1주차 — 수업 시작 전 반드시 알아야 할 것

AI 시대의 코딩 윤리(Coding Ethics): "한 단계 더 들어가기"

토론 수업(Discussion-Based Class) 강의안

---

0. 수업 정보(Class Information)

항목(Item)	내용(Description)
목표(Objective)	AI 코딩 도구 사용 시 "한 단계 더 들어가기"의 필요성을 체득한다
핵심 질문(Key Question)	"AI가 만든 코드를 그대로 쓰면 왜 위험한가?"
준비물(Materials)	Google Colab, 본 자료

---

1. 도입(Introduction): 두 학생의 과제 (5분)

1.1 상황 제시(Scenario)

학기 말, 동일한 과제를 받은 두 학생이 있다. 과제는 "코로나19(COVID-19) 확진자 데이터를 분석하여 증가 추세를 시각화하라"이다.

🔵 학생 A의 제출물(Student A's Submission)

[사용한 프롬프트](Prompt)

```
코로나19 확진자 데이터 분석하고 그래프 그려줘
```

[제출한 코드](Submitted Code) — AI가 생성한 코드를 그대로 복사·붙여넣기(Copy & Paste)

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('covid_data.csv')
```

```
df['date'] = pd.to_datetime(df['date'])
df.plot(x='date', y='cases', figsize=(12,6))
plt.title('COVID-19 Cases')
plt.show()
```

[제출한 분석**]**(Submitted Analysis)

> "코로나19 확진자 수는 시간이 지남에 따라 증가하는 추세를 보인다. 그래프에서 확인할 수 있듯이 팬데믹 초기에 급격한 증가가 있었다."

🟢 학생 **B**의 제출물**(Student B's Submission)**

[사용한 프롬프트**]**(Prompt)

```
 pandas로 CSV 파일을 읽어서 'date' 열을 datetime으로 변환하고,
'cases' 열의 7일 이동평균(rolling mean, window=7)을 계산하는 코드를 작성해줘.
x축은 날짜, y축은 원본 확진자 수(연한 파란색, alpha=0.3)와
이동평균(진한 파란색, linewidth=2)을 하나의 그래프에 겹쳐 그리고 싶다.
figsize는 (14, 6)으로 해줘.
```

[제출한 코드**]**(Submitted Code) — AI가 생성한 코드를 검증(Verification)하고 수정(Modification)

```
 import pandas as pd
import matplotlib.pyplot as plt

# 데이터 불러오기(Load Data)
df = pd.read_csv('covid_data.csv')
df['date'] = pd.to_datetime(df['date'])

# 7일 이동평균 계산(7-Day Rolling Mean)
df['cases_7day_avg'] = df['cases'].rolling(window=7).mean()

# 검증: 이동평균이 제대로 계산되었는지 확인(Verification)
print(f"원본 데이터 행 수: {len(df)}")
print(f"이동평균 NaN 개수: {df['cases_7day_avg'].isna().sum()}")  # 처음 6개는 NaN이어야 함
print(f"최대 확진자 수: {df['cases'].max():,.0f}")
print(f"최대 확진자 발생일: {df.loc[df['cases'].idxmax(), 'date'].strftime('%Y-%m-%d')}")

# 시각화(Visualization)
fig, ax = plt.subplots(figsize=(14, 6))
ax.bar(df['date'], df['cases'], color='steelblue', alpha=0.3, label='일별 확진자(Daily)')
ax.plot(df['date'], df['cases_7day_avg'], color='darkblue', linewidth=2, label='7일 이동
ax.set_title('COVID-19 일별 확진자 수 및 7일 이동평균')
```

```
ax.set_xlabel('날짜(Date)')
ax.set_ylabel('확진자 수(Cases)')
ax.legend()
plt.tight_layout()
plt.show()
```

[제출한 분석**]**(Submitted Analysis)

> "일별 확진자 수(막대, 연한 파란색)는 노이즈(Noise)가 심하여 추세 파악이 어려우나, 7일 이동평균(실선, 진한 파란색)을 적용하면 **3**개의 뚜렷한 피크(Peak)가 관찰된다. 첫 번째 피크는 2020년 8월(약 350명/일), 두 번째는 2020년 12월(약 1,100명/일), 세 번째는 2022년 3월(약 620,000명/일)이다. 세 번째 피크가 압도적으로 큰 이유는 오미크론(Omicron) 변이의 높은 전파력 때문으로 판단된다. 이동평균의 NaN 6개는 window=7 설정에 의한 정상적인 결과임을 확인하였다."

1.2 질문 던지기**(Opening Question)**

> **"**두 학생 모두 **AI**를 사용했다**.** 둘 다 코드가 돌아가고**,** 그래프가 나온다**.** 그런데 왜 학생 **B**의 제출물이 더 좋은가**?** 무엇이 다른가**?"**

학생들에게 1분간 생각할 시간을 준 뒤, 2~3명에게 의견을 묻는다.

---

2. 핵심 개념**(Core Concept): "**한 단계 더 들어가기**" (10**분**)**

2.1 AI 사용의 3단계 모델**(Three-Level Model)**

AI를 사용하여 코딩하는 행위는 다음 **3**단계(Level)로 구분할 수 있다.

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│   Level 0: 통째로 맡기기(Full Delegation)                    │
│   ────────────────────────────────────                       │
│   "코로나 데이터 분석해줘"                                   │
│   → AI가 문제 해석 + 코드 작성 + 결과 해석까지 전부 수행     │
│   → 학생은 Ctrl+C, Ctrl+V만 수행                             │
│   → ❌ 학습 효과 없음(No Learning)                           │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                                                              │
```

```
│   Level 1: 구현만 맡기기(Implementation Delegation)          │
│   ────────────────────────────────────────                    │
│   "7일 이동평균을 계산하는 코드를 작성해줘,                     │
│    window=7, rolling mean 사용"                                │
│   → 학생이 '무엇을 할지'를 결정, AI가 '어떻게'를 구현          │
│   → ⚠️ 코드는 나오지만, 결과 검증이 빠져 있다                  │
│                                                                │
├────────────────────────────────────────────────────────────────┤
│                                                                │
│   Level 2: 한 단계 더 들어가기(One Step Deeper)      ✅        │
│   ────────────────────────────────────────                    │
│   ① 조건 설정(Condition Setting):                              │
│      "window=7, 처음 6개는 NaN이어야 한다"                     │
│   ② 구현 요청(Implementation Request):                         │
│      "rolling mean으로 계산해줘"                               │
│   ③ 검증(Verification):                                        │
│      "NaN이 6개 맞는지 확인", "최댓값 날짜 확인"               │
│   ④ 관찰(Observation):                                         │
│      "피크가 3개, 세 번째가 가장 크다 → 오미크론 때문"         │
│   → 학생이 사고의 주인(Owner of Thinking)                      │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

2.2 왜 "한 단계 더"가 필요한가?

AI가 생성한 코드에는 다음과 같은 함정(Trap)이 존재한다.

함정**(Trap)**	설명**(Description)**	실제 사례**(Example)**
그럴듯한 오류 (Plausible Error)	코드가 실행되지만 결과가 틀리다	`rolling(7)` 대신 `rolling(7, center=True)` 를 쓰면 미래 데이터가 포함된다
맥락 무시(Context Ignorance)	AI는 데이터의 맥락을 모른다	결측값(Missing Value)이 있는 날의 이동평균이 왜곡될 수 있다
일반론 생성(Generic Output)	AI는 "보통 이렇다"를 출력한다	"확진자 수가 증가 추세이다"는 어떤 전염병 데이터에나 적용 가능한 문장이다
검증 불가 (Unverifiable)	실행 흔적이 없으면 맞는지 틀린지 알 수 없다	그래프 없이 "3개의 피크가 있다"고 쓰면, 실제로 본 건지 AI가 만든 건지 구분 불가

2.3 한 줄 원칙**(One-Line Principle)**

> **"AI**를 쓸수록**,** 나는 한 단계 더 들어가서 검증해야 한다**."**

이것은 단순한 윤리(Ethics) 문제가 아니다. 실력(Competence)의 문제이다. AI가 만든 코드를 검증할 수 없는 사람은, AI 없이도 코딩할 수 없고, AI와 함께해도 올바른 결과를 낼 수 없다.

---

3. 토론 활동(Discussion Activity): "이 프롬프트, 괜찮은가?" (10분)

3.1 토론 규칙(Discussion Rules)

- 2~3명이 한 조(Team)를 이룬다.
- 각 사례(Case)를 읽고, 허용(OK) / 경고(Warning) / 부정행위(Misconduct) 중 하나로 판정한다.
- 판정 근거를 한 문장으로 적는다.
- 조별로 결과를 발표하고, 다른 조와 비교한다.

3.2 판정 기준표(Evaluation Criteria) — 학생에게 배포

```
┌──────────────────────────────────────────────────────────────┐
│                    평가 체크리스트(Checklist)                  │
│                                                              │
│  □ 프롬프트에 조건/제약이 명시되어 있는가?                      │
│    (예: seed값, 반복 횟수, 변수명, 파라미터)                    │
│                                                              │
│  □ 실행 흔적(코드/출력/그래프)이 제출물에 있는가?               │
│                                                              │
│  □ 관찰이 구체적인가?                                          │
│    (위치·수치·패턴이 포함되어 있는가?)                          │
│                                                              │
│  □ 검증 행위가 있는가?                                         │
│    ("이 값이 맞는지 확인했다"는 흔적)                           │
│                                                              │
│  □ 설명이 일반론으로만 끝나지 않는가?                           │
│                                                              │
│  ──────────────────────────────────────                      │
│  2개 이상 ✅ → 허용(OK)                                        │
│  1개만 ✅   → 경고(Warning): 보완 필요                          │
│  0개 ✅     → 부정행위(Misconduct) 의심                         │
└──────────────────────────────────────────────────────────────┘
```

3.3 토론 사례(Discussion Cases)

【사례 **1**】 뉴욕 에어비앤비(**Airbnb**) 숙박료 예측

프롬프트**:**

```
 뉴욕에서 방이 둘 딸린 집을 에어비앤비에 내놓으려 한다.
적당한 숙박료를 구해줘.
```

제출물**:**

> "뉴욕의 2베드룸 에어비앤비 평균 숙박료는 약 $150~$200이다. 맨해튼은 더 비싸고, 브루클린은 상대적으로 저렴하다."

토론 질문**:** 이 제출물은 허용/경고/부정행위 중 어디에 해당하는가? 이유는?

【사례 **2**】 **LoL(League of Legends)** 승리 공식 분석

프롬프트**:**

```
 LoL 랭크 게임 데이터에서
승리팀과 패배팀의 첫 번째 타워(firstTower),
첫 번째 용(firstDragon), 첫 번째 바론(firstBaron)의
획득 비율을 각각 계산하는 코드를 작성해줘.
데이터는 games.csv이고, winner 열이 1이면 블루팀 승리, 2이면 레드팀 승리이다.
결과는 막대그래프(bar chart)로 비교해줘.
```

제출물**:**

```
 # [학생이 작성한 검증 코드]
print(f"전체 게임 수: {len(df)}")
print(f"블루팀 승률: {(df['winner']==1).mean():.1%}")
print(f"첫 타워 획득 시 승률: {win_rate_first_tower:.1%}")
```

> "분석 결과, 첫 번째 타워를 획득한 팀의 승률은 64.3%이다. 첫 번째 용을 획득한 팀의 승률은 59.1%, 바론은 72.8%이다. 따라서 바론 획득이 승리에 가장 큰 영향을 미치며, 이는 바론 버프(Baron Buff)의 팀 전투 이점과 일치한다."

토론 질문**:** 이 제출물은 허용/경고/부정행위 중 어디에 해당하는가?

【사례 **3**】 중고 자동차**(Used Car)** 가격 예측

프롬프트**:**

```
 중고차 가격 예측 머신러닝 모델 만들어줘.
대학생 수준으로 설명도 붙여줘.
```

제출물**:**

> (AI가 생성한 긴 설명문 + 코드 전체를 그대로 복사)
>
> "랜덤 포레스트(Random Forest) 모델을 사용하여 중고차 가격을 예측하였다. R^2 스코어는 0.87로 높은 예측 성능을 보인다. 특성 중요도(Feature Importance)를 확인하면 연식(year)이 가장 중요한 변수이며, 이는 자동차의 감가상각과 관련이 있다."

토론 질문**:** $R^2 = 0.87$이라는 숫자가 있으니 "구체적"인가?

【사례 **4**】 코로나**19(COVID-19)** 데이터 직접 분석

프롬프트**:**

```
 covid_data.csv 파일에서
1) 'date' 열을 datetime으로 변환,
2) 'new_cases' 열의 7일 이동평균(rolling, window=7) 계산,
3) 2020년 1월 ~ 2022년 12월 범위만 필터링,
4) x축=날짜, y축=이동평균 그래프 그려줘.
그래프에 한국(South Korea)과 일본(Japan)을 겹쳐 그리고,
범례(legend)를 넣어줘.
```

제출물**:**

```
 # 검증(Verification)
print(f"한국 데이터 행 수: {len(df_kr)}")
print(f"일본 데이터 행 수: {len(df_jp)}")
print(f"한국 최대 일일 확진: {df_kr['new_cases'].max():,.0f}명 "
      f"({df_kr.loc[df_kr['new_cases'].idxmax(), 'date'].strftime('%Y-%m-%d')})")
print(f"일본 최대 일일 확진: {df_jp['new_cases'].max():,.0f}명 "
      f"({df_jp.loc[df_jp['new_cases'].idxmax(), 'date'].strftime('%Y-%m-%d')})")
```

> "한국의 최대 일일 확진자 수는 2022년 3월 17일 약 621,328명이며, 일본은 2022년 8월 19일 약 261,029명이다. 한국의 오미크론 피크가 일본보다 약 5개월 앞서 발생했으며, 피크 규모도 약 2.4배 크다(621,328 / 261,029 ≈ 2.38). 이동평균 그래프에서 한국은 급격

> 한 상승-하강 패턴을 보이는 반면, 일본은 상대적으로 완만한 여러 번의 파동(Wave)이 관찰된다."

토론 질문**:** 이 제출물의 어떤 점이 "한 단계 더 들어간" 것인가?

【사례 **5**】 심부전증**(Heart Failure)** 예측

프롬프트**:**

```
심부전증 예측해줘
```

제출물**:**

> "심부전증 예측 모델의 정확도는 약 85%이다."

토론 질문**:** 이 제출물의 문제점을 3가지 이상 찾아보라.

---

4. 정리(Wrap-Up): 이 수업의 평가 원칙 (5분)

4.1 본 수업의 AI 사용 규칙(AI Usage Policy)

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║   본 수업에서 AI 사용은 금지(Prohibition)가 아니다.            ║
║   단, 다음 원칙을 따라야 한다.                                 ║
║                                                               ║
║   1. 프롬프트를 함께 제출하라.                                 ║
║      → "어떤 질문을 했는가"가 평가 대상이다.                    ║
║                                                               ║
║   2. 실행 흔적을 남겨라.                                       ║
║      → 코드, 출력(Output), 그래프가 있어야 한다.                ║
║                                                               ║
║   3. 검증하라.                                                 ║
║      → "이 결과가 맞는지 어떻게 확인했는가?"를 적어라.          ║
║                                                               ║
║   4. 구체적으로 관찰하라.                                       ║
║      → "증가 추세이다" (✗) vs "3월 17일 621,328명 피크" (✓)     ║
║                                                               ║
║   요약: AI를 쓸수록, 한 단계 더 들어가라.                       ║
```

```
╚═══════════════════════════════════════════════════════════════════════╝
```

4.2 학기 중 적용 방법(How It Applies)

평가 항목(Evaluation Item)	적용 방식(Application)
매주 연습문제(Weekly Exercises)	프롬프트 제출 선택사항, 단 코드+출력+관찰 필수
과제(Assignments)	프롬프트 제출 필수 + 검증 코드 포함 필수
중간/기말고사(Exams)	**AI** 사용 금지 — 본인의 실력만으로 해결
프로젝트(Project)	AI 사용 내역 명시 + 본인 기여분(Contribution) 표시

4.3 기억할 한 문장(Takeaway)

> "**AI**는 도구(Tool)이다**.** 도구를 잘 쓰는 사람은 도구의 결과를 검증할 수 있는 사람이다**."**

---

5. 토론 워크시트(Discussion Worksheet) — 학생 배포용

워크시트**: "**이 프롬프트**,** 괜찮은가**?"**

이름(Name)**: __** 조**(Team): __**

Q1. 사례 **1 (**에어비앤비**)**

항목	판정
나의 판정(My Verdict)	☐ 허용 ☐ 경고 ☐ 부정행위
판정 근거(한 문장)	

Q2. 사례 **2 (LoL** 승리 공식**)**

항목	판정
나의 판정(My Verdict)	☐ 허용 ☐ 경고 ☐ 부정행위
판정 근거(한 문장)	

Q3. 사례 **3 (**중고 자동차**)**

항목	판정
나의 판정(My Verdict)	☐ 허용 ☐ 경고 ☐ 부정행위
판정 근거(한 문장)	
R^2 = 0.87이 "구체적"이라고 볼 수 있는가? 이유는?	

Q4. 사례 **4 (**코로나**19)**

항목	판정
나의 판정(My Verdict)	☐ 허용 ☐ 경고 ☐ 부정행위
"한 단계 더 들어간" 부분을 3가지 이상 적어라	

Q5. 사례 **5 (**심부전증**)**

항목	판정
나의 판정(My Verdict)	☐ 허용 ☐ 경고 ☐ 부정행위
문제점 3가지 이상	

Q6. 나의 다짐**(My Commitment)**

> "이번 학기 AI를 사용할 때, 나는 _____ 하겠다."

---

7. 연습문제**(Exercises) — 10**문항 **+** 해답**(Solutions)**

> 아래 연습문제는 1주차 파이썬 기초 학습 후, AI 사용 원칙을 직접 적용해 보는 문제이다.
> 모든 문제에서 프롬프트 **+** 코드 **+** 검증 **+** 관찰을 함께 작성해야 한다.

문제 **1.** 좋은 프롬프트 **vs** 나쁜 프롬프트 구별**(Prompt Evaluation)**

다음 두 프롬프트 중 어느 것이 더 좋은 프롬프트인지 판별하고, 이유를 설명하라.

프롬프트 **A:**

```
파이썬으로 리스트 정렬해줘
```

프롬프트 **B:**

```
정수 리스트 [64, 34, 25, 12, 22, 11, 90]을
버블 정렬(Bubble Sort) 알고리즘으로 오름차순 정렬하는 코드를 작성해줘.
각 패스(pass)마다 리스트 상태를 출력해서 정렬 과정을 확인할 수 있게 해줘.
```

📝 해답**(Solution)**
프롬프트 B가 더 좋은 프롬프트이다. | 평가 기준 | 프롬프트 A | 프롬프트 B | |---|---|---| | 조건 명시 | ✗ (어떤 리스트? 어떤 알고리즘?) | ✓ (구체적 리스트, 버블 정렬 지정) | | 검증 가능성 | ✗ (결과 확인 방법 없음) | ✓ (각 패스마다 출력하여 과정 확인) | | 학습 효과 | ✗ (결과만 받음) | ✓ (정렬 과정을 관찰할 수 있음) | 프롬프트 B는 (1) 입력 데이터가 명시되어 있고, (2) 알고리즘이 지정되어 있으며, (3) 중간 과정을 출력하여 검증할 수 있다.

문제 **2.** 변수**(Variable)** 활용 **—** 검증 코드 작성

다음 코드의 실행 결과를 손으로 먼저 예측한 뒤, 실제로 실행하여 검증하라.

```
x = 10
y = 3
a = x // y
b = x % y
c = x ** y
print(f"몫: {a}, 나머지: {b}, 거듭제곱: {c}")
```

(1) 손으로 예측한 결과를 적어라. (2) 실행 결과와 비교하여 일치하는지 확인하라. (3) 만약 `x = -10, y = 3` 이면 결과가 어떻게 달라지는지 예측하고 검증하라.

📝 해답**(Solution)**
(1) 손으로 예측: - `x // y` = 10 // 3 = 3 (몫) - `x % y` = 10 % 3 = 1 (나머지) - `x ** y` = 10^3 = 1000 (거듭제곱) - 출력: `몫: 3, 나머지: 1, 거듭제곱: 1000` **(2) 실행 결과:** 예측과 일치한다. **(3) x = -10, y = 3인 경우:** - `-10 // 3` = **-4** (파이썬은 음의 무한대 방향으로 내림한다, -3.33... → -4) - `-10 % 3` = **2** (-10 = (-4) × 3 + 2) - `-10 ** 3` = **-1000** - 출력: `몫: -4, 나머지: 2, 거듭제곱: -1000` **검증 포인트:** 파이썬의 `//` 연산자는 **내림**(Floor Division)**이므로**, 음수에서는 C/Java와 결과가 다르다. 이것이 "한 단

계 더 들어가기"이다 — AI가 코드를 줘도, 음수 입력에서 어떻게 동작하는지 직접 확인해야 한다.

문제 **3.** 문자열**(String) — AI** 결과 검증

AI에게 다음 프롬프트를 보내고, 결과를 검증하라.

프롬프트**:**

```
 문자열 "Hello, World! Hello, Python!"에서
"Hello"가 몇 번 나타나는지 세는 3가지 방법의 코드를 작성해줘.
(count 메서드, split 활용, for문 활용)
```

(1) AI가 생성한 코드를 실행하라. (2) 3가지 방법의 결과가 모두 같은지 확인하라. (3) 만약 `"hello"` (소문자)도 포함하여 세려면 어떻게 수정해야 하는지 직접 작성하라.

📝 해답**(Solution)**

```
 text = "Hello, World! Hello, Python!"

# 방법 1: count() 메서드
count1 = text.count("Hello")

# 방법 2: split() 활용
count2 = len(text.split("Hello")) - 1

# 방법 3: for문 활용
count3 = 0
for i in range(len(text) - len("Hello") + 1):
    if text[i:i+len("Hello")] == "Hello":
        count3 += 1

# 검증: 세 결과가 같은지 확인
print(f"count(): {count1}, split(): {count2}, for문: {count3}")
print(f"모두 같은가? {count1 == count2 == count3}")  # True
```

(3) 대소문자 무시(Case-Insensitive) **버전:**

```
 text_lower = text.lower()
count_all = text_lower.count("hello")
print(f"대소문자 무시 결과: {count_all}")  # 2
```

검증 포인트: AI가 준 3가지 방법이 모두 같은 결과를 내는지 **교차 검증**(Cross-Validation)**하는** 것이 "한 단계 더 들어가기"이다.

문제 **4.** 프롬프트 개선**(Prompt Improvement)**

다음의 나쁜 프롬프트를 **Level 2** 수준으로 개선하라.

원래 프롬프트**:**

```
코로나 확진자 수를 리스트에 저장하고 최대값 구해줘
```

조건: 개선된 프롬프트에는 (1) 구체적 데이터, (2) 변수명, (3) 검증 방법이 포함되어야 한다.

📝 해답**(Solution)**
개선된 프롬프트 예시:

```
 2022년 1월 첫째 주(1일~7일) 한국의 일별 코로나19 신규 확진자 수가
다음과 같다: [4,561, 3,865, 3,024, 4,283, 4,542, 3,128, 3,347]

이 데이터를 daily_cases라는 리스트에 저장하고,
1) 최대 확진자 수와 해당 날짜(1월 며칠인지),
2) 최소 확진자 수와 해당 날짜,
3) 7일 평균 확진자 수를
구하는 코드를 작성해줘.

날짜는 1월 1일부터 시작이다. (인덱스 0 = 1월 1일)
결과를 f-string으로 출력해줘.
```

개선 포인트: | 항목 | 원래 | 개선 후 | |---|---|---| | 데이터 | 불명 | 구체적 수치 7개 | | 변수명 | 없음 | `daily_cases` 지정 | | 검증 | 없음 | 날짜 역추적 가능 (인덱스 = 날짜) | | 출력 형식 | 없음 | f-string 지정 |

문제 **5.** 리스트**(List) —** "한 단계 더" 실습

다음 코드를 AI에게 요청하여 받은 후, **3**가지 검증을 수행하라.

과제**:** 1부터 100까지의 정수 중 3의 배수이면서 5의 배수가 아닌 수를 리스트로 만들어라.

(1) AI가 생성한 코드를 실행하라. (2) 리스트의 원소 개수를 예측하고 검증하라. (3) 리스트의 첫 5개와 마지막 5개 원소를 출력하여 확인하라. (4) 15, 30, 45가 리스트에 포함되지 않았는지 확인하라 (이들은 3과 5의 공배수이다).

📝 해답**(Solution)**

```
 # 코드
result = [x for x in range(1, 101) if x % 3 == 0 and x % 5 != 0]

# 검증 1: 원소 개수
```

```
print(f"원소 개수: {len(result)}")  # 27개

# 검증 2: 첫 5개와 마지막 5개
print(f"첫 5개: {result[:5]}")      # [3, 6, 9, 12, 18]
print(f"마지막 5개: {result[-5:]}")  # [84, 87, 93, 96, 99]

# 검증 3: 15, 30, 45가 포함되지 않았는지
for num in [15, 30, 45, 60, 75, 90]:
    assert num not in result, f"{num}이 포함되어 있다!"
print("검증 통과: 3과 5의 공배수는 모두 제외되었다.")
```

수동 검증: - 1~100에서 3의 배수: 33개 (3, 6, 9, ..., 99) - 그 중 15의 배수(3과 5의 공배수): 6개 (15, 30, 45, 60, 75, 90) - 따라서 33 - 6 = **27개** ✓

문제 **6.** 자료형**(Data Type)** 함정 찾기**(Trap Detection)**

AI에게 "사용자 입력을 받아서 두 수의 합을 구하는 코드"를 요청하면, 다음과 같은 코드를 줄 수 있다.

```
a = input("첫 번째 숫자: ")
b = input("두 번째 숫자: ")
print(f"합계: {a + b}")
```

(1) 이 코드를 실행하고, `3` 과 `5` 를 입력했을 때 결과를 확인하라. (2) 예상과 다른 결과가 나왔다면, 원인을 설명하라. (3) 올바른 코드로 수정하라.

📝 해답**(Solution)**
(1) 실행 결과: `합계: 35` (8이 아니라 35가 출력된다!) **(2) 원인:** `input()` 함수는 항상 **문자열**(str)**을** 반환한다. 따라서 `"3" + "5"`는 문자열 연결(Concatenation)이 되어 `"35"`가 된다. **(3) 수정된 코드:**

```
a = int(input("첫 번째 숫자: "))
b = int(input("두 번째 숫자: "))
print(f"합계: {a + b}")  # 8

# 추가 검증: 소수점 입력 대비
# a = float(input("첫 번째 숫자: "))
```

교훈: AI가 준 코드가 "돌아간다"고 "맞는" 것이 아니다. `input()`의 반환 타입을 확인하는 것이 "한 단계 더 들어가기"이다.

문제 **7.** 딕셔너리**(Dictionary) —** 데이터 기반 관찰

다음 데이터는 서울시 5개 구(District)의 2023년 코로나19 누적 확진자 수이다.

```
covid_seoul = {
    "강남구": 185432,
    "송파구": 172891,
    "강서구": 168234,
    "관악구": 98765,
    "종로구": 67543
}
```

(1) 확진자가 가장 많은 구와 가장 적은 구를 구하는 코드를 작성하라. (2) 전체 평균 확진자 수를 구하라. (3) 평균 이상인 구와 평균 미만인 구를 각각 리스트로 만들어라. (4) 결과를 보고 "구체적 관찰" 한 문장을 작성하라 (일반론이 아닌, 이 데이터에서만 보이는 것).

📝 해답(Solution)

```
covid_seoul = {
    "강남구": 185432, "송파구": 172891, "강서구": 168234,
    "관악구": 98765, "종로구": 67543
}

# (1) 최대/최소
max_gu = max(covid_seoul, key=covid_seoul.get)
min_gu = min(covid_seoul, key=covid_seoul.get)
print(f"최다: {max_gu} ({covid_seoul[max_gu]:,}명)")
print(f"최소: {min_gu} ({covid_seoul[min_gu]:,}명)")

# (2) 평균
avg = sum(covid_seoul.values()) / len(covid_seoul)
print(f"평균: {avg:,.0f}명")

# (3) 평균 이상/미만
above = [gu for gu, cases in covid_seoul.items() if cases >= avg]
below = [gu for gu, cases in covid_seoul.items() if cases < avg]
print(f"평균 이상: {above}")
print(f"평균 미만: {below}")
```

출력:

```
최다: 강남구 (185,432명)
최소: 종로구 (67,543명)
평균: 138,573명
평균 이상: ['강남구', '송파구', '강서구']
평균 미만: ['관악구', '종로구']
```

(4) 구체적 관찰 예시: > "강남구(185,432명)와 종로구(67,543명)의 차이는 약 2.7배이다. > 평균(138,573명) 이상인 3개 구는 모두 강남 3구(강남·송파·강서) 권역에 해당하며, > 이는 해당 지역의 높은 인구 밀도 및 유동인구와 관련이 있을 수 있다." **나쁜 관찰 예시 (일반

론):** > "코로나 확진자 수는 구마다 차이가 있다." → ❌ 이 문장은 어떤 도시 데이터에도 붙일 수 있다.

문제 **8. f-string** 검증 실습**(Verification Practice)**

다음 코드의 출력을 먼저 손으로 예측한 뒤, 실행하여 검증하라.

```
name = "Python"
version = 3.12
year = 2024

print(f"{name}의 최신 버전은 {version}이다. ({year}년 기준)")
print(f"{'=' * 30}")
print(f"{'Python':>15}")
print(f"{'Python':^15}")
print(f"{'Python':<15}")
print(f"원주율: {3.141592:.2f}")
print(f"큰 수: {1234567890:,}")
print(f"퍼센트: {0.8567:.1%}")
```

📝 해답**(Solution)**

```
Python의 최신 버전은 3.12이다. (2024년 기준)
==============================
         Python
     Python
Python
원주율: 3.14
큰 수: 1,234,567,890
퍼센트: 85.7%
```

검증 포인트: - `>15`: 15칸 안에서 오른쪽 정렬(Right-Align) - `^15`: 15칸 안에서 가운데 정렬(Center-Align) - `<15`: 15칸 안에서 왼쪽 정렬(Left-Align) - `:.2f`: 소수점 이하 2자리 - `:,`: 천 단위 쉼표 - `:.1%`: 소수를 퍼센트로 변환 (소수점 1자리) **"한 단계 더":** `{0.8567:.1%}`가 `85.7%`가 되는 이유를 설명할 수 있는가? 0.8567 × 100 = 85.67 → 소수점 1자리 반올림 → 85.7%이다.

문제 **9.** 프롬프트 작성 실습**(Prompt Writing Practice)**

다음 과제를 수행하기 위한 **Level 2** 프롬프트를 직접 작성하라. (코드를 작성하는 것이 아니라, **AI**에게 보낼 프롬프트를 작성하는 문제이다.)

과제**:** "뉴욕 에어비앤비 데이터에서 맨해튼(Manhattan)의 2베드룸(Bedroom) 숙소 가격 분포를 분석하라."

프롬프트에 반드시 포함해야 하는 요소: - 데이터 파일명과 열(Column) 이름 - 필터링 조건 - 이상치(Outlier) 처리 기준 - 출력 형식 (통계량 + 그래프 종류) - 검증 방법

📝 해답**(Solution)**

모범 프롬프트 예시:

```
AB_NYC_2019.csv 파일에서 다음 조건으로 데이터를 필터링해줘:
- neighbourhood_group == 'Manhattan'
- bedrooms == 2 (결측값 제외)
- price > 0 and price <= 1000 (이상치 제외)

필터링된 데이터에 대해:
1) 기술 통계량 출력 (count, mean, median, std, min, max)
2) 가격 분포 히스토그램(histogram, bins=50, color='steelblue')
3) 동네(neighbourhood)별 평균 가격 상위 10개를 수평 막대그래프(barh)로

검증:
- 필터링 전후 데이터 행 수를 출력해서 몇 % 제거되었는지 확인
- median과 mean의 차이가 크면 왜 그런지 주석으로 설명
```

체크리스트 확인: - ✅ 파일명, 열 이름 명시 - ✅ 필터 조건 구체적 (지역, 방 수, 가격 범위) - ✅ 이상치 기준 (price > 1000 제외) - ✅ 출력 형식 (통계량 + 히스토그램 + 막대그래프) - ✅ 검증 방법 (전후 비교, mean vs median)

문제 **10.** 종합 **— AI** 결과 비판적 평가**(Critical Evaluation)**

AI에게 "1부터 100까지의 소수(Prime Number)를 구하는 코드"를 요청했더니 다음 코드를 주었다.

```
primes = []
for num in range(1, 101):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            primes.append(num)
print(f"소수의 개수: {len(primes)}")
print(f"소수 목록: {primes}")
```

(1) 이 코드를 실행하고, 소수의 개수가 **25**개인지 확인하라. (2) 이 코드의 비효율성(Inefficiency)을 찾고, 개선 방안을 제시하라. (3) 개선된 코드를 작성하고, 두 코드의 실행 시간을 비교하라. (4) `range(2, num)` 대신 `range(2, int(num**0.5) + 1)` 로 바꾸면 왜 더 빠른지 설명하라.

📝 해답(Solution)

(1) 실행 결과: 25개 ✓ (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97) **(2) 비효율성:** - `range(2, num)`: 97이 소수인지 확인하려면 2부터 96까지 **95번** 나눠봐야 한다. - 실제로는 √97 ≈ 9.8이므로, 2부터 9까지 **8번**만 확인하면 충분하다. **(3) 개선된 코드 + 시간 비교:**

```
import time

# 원래 코드
start = time.time()
primes1 = []
for num in range(1, 101):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            primes1.append(num)
time1 = time.time() - start

# 개선된 코드
start = time.time()
primes2 = []
for num in range(2, 101):
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            break
    else:
        primes2.append(num)
time2 = time.time() - start

# 검증
print(f"결과 동일: {primes1 == primes2}")
print(f"원래 코드: {time1:.6f}초")
print(f"개선 코드: {time2:.6f}초")
```

(4) 이유 설명: 합성수(Composite Number) n = a × b에서, a와 b 중 하나는 반드시 √n 이하이다. 따라서 √n까지만 나눗셈을 확인하면, 소수 여부를 판별할 수 있다. 97의 경우 √97 ≈ 9.8이므로 2, 3, 4, 5, 6, 7, 8, 9만 확인하면 된다. **교훈:** AI가 "정답"을 주더라도, **효율성(Efficiency)**을 따지는 것**이 "한 단계 더 들어가기"이다. AI는 "돌아가는 코드"를 주지만, "좋은 코드"를 주는 것은 아닐 수 있다.

부록(Appendix): 빠른 참조표(Quick Reference)

A. 프롬프트 품질 체크리스트(Prompt Quality Checklist)

체크 항목(Check Item)	나쁜 예(Bad)	좋은 예(Good)
데이터 명시	"데이터 분석해줘"	"covid_data.csv의 'new_cases' 열"
조건 명시	"필터링해줘"	"price > 0 and price <= 1000"
변수명 지정	(없음)	"daily_cases라는 리스트에 저장"
출력 형식	"그래프 그려줘"	"히스토그램, bins=50, color='steelblue'"
검증 방법	(없음)	"필터링 전후 행 수를 출력"

B. "한 단계 더" 체크리스트(One Step Deeper Checklist)

단계 (Step)	질문(Question)	행동(Action)
1. 실행 전	"이 코드가 무엇을 하는지 설명할 수 있는가?"	주요 함수/메서드의 역할을 주석으로 적기
2. 실행 후	"결과가 맞는지 어떻게 아는가?"	검증 코드 추가 (건수, 범위, 교차 확인)
3. 관찰	"이 데이터에서만 보이는 것은 무엇인가?"	구체적 수치/패턴/비교를 문장으로 작성
4. 확장	"입력이 달라지면 어떻게 되는가?"	엣지 케이스(Edge Case) 테스트

본 자료는 *1*주차 수업의 도입부*(*약 *25~30*분*)*에 사용하며, 이후 *AI* 개론 및 파이썬 기초 수업으로 이어진다*.*

참고*: AI* 시대의 학습 윤리 *v2 (2026)*